

ELMAR
CNC-Cutter

Autoren: Alexandra Roth 7025637
David Atrops 7013979
Dennis Smirnow 7023434
Leon Ronge 7026641
Samuel Drees 7026354

Studiengang: Maschinenbau und Design

Erstprüfer: Prof. Dr. Elmar Wings
Malena Wilken

Abgabedatum: 10. Mai 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	5
I. Domain Knowledge - Application	7
II. Domain Knowledge - Hardware	9
1. Sensor: Endschalter	11
1.1. Einleitung: Positionssensoren	11
1.2. Definition und Funktion: Elektromechanischer Endschalter	11
1.3. Specification	11
1.4. Verwendung und Schaltfunktionen	12
1.5. Analyse: Vor- und Nachteile	13
1.6. Ausblick: Intelligente und vernetzte Sensorik	14
1.7. Library	14
1.8. Beispiel	14
1.8.1. Manual	14
1.8.2. Erklärung des Beispiel-Codes	14
1.9. Justage	15
1.10. Tests	15
1.10.1. Einfacher Funktions-Test	15
1.10.2. Manual	16
1.10.3. Einfacher Test-Code	16
1.11. Einfache Anwendung	16
1.11.1. Manual	17
1.12. Weiterführende Literatur	18
2. Druckknopf	19
2.1. General	19
2.2. Joy IT	19
2.3. Spezifikationen	19
2.3.1. Functions	19
2.4. Beispiel Code	19
2.4.1. Manual	19
2.4.2. Code	20
2.5. Einfacher Testcode	20
2.5.1. Manual	20
2.5.2. Code	21
2.6. Einfache Anwendung	21
2.6.1. Manual	21
2.6.2. Code	22
2.7. Further Readings	24

III. Domain Knowledge - Tools	25
IV. Developmenmt	27
V. Monitoring	29
VI. Verification - Evaluation - Conclusion	31
VII.Anhang	33
Literaturverzeichnis	35
Literaturverzeichnis	35

Abbildungsverzeichnis

1.1. Darstellung des im Projekt verwendeten Endschalters	12
1.2. Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO	13

Teil I.

Domain Knowledge - Application

Teil II.

Domain Knowledge - Hardware

1. Sensor: Endschalter

1.1. Einleitung: Positionssensoren

In der industriellen Fertigungssteuerung nimmt die Erfassung von Zuständen eine zentrale Rolle ein. Gemäß der Fachliteratur wandelt ein Sensor (lat. sensus, der Sinn) „eine physikalische Größe (z. B. Kraft oder Temperatur) mit Hilfe eines physikalischen Effekts in ein elektrisches Signal um“ [Her+12, S. 433]. Speziell im Maschinenbau wird bei der Steuerung zwischen zwei grundlegenden Messverfahren unterschieden: Der „Positionserfassung durch Schalter“ und der „Positionserfassung durch Wegbeobachtung“. [Her+12, S. 435]

Bevor der Fokus auf kontaktbehaftete Schalter gelegt wird, sind folgende berührungslose Sensoren (Sensorschalter) als moderner Industriestandard zu nennen:

- Induktive Sensoren: Erfassen metallische Elemente „ohne Kontakt zwischen der Bewegung und dem Sensor“ [Her+12, S. 436].
- Kapazitive Sensoren: Nutzen die Beeinflussung eines elektrischen Feldes zur Kapazitätsänderung [Her+12, S. 438].
- Optische Sensoren: Arbeiten mit Infrarot-Lichtstrahlen zur Objekterkennung (z. B. Lichttaster oder Lichtschranken) [Her+12, S. 440].
- Ultraschall-Sensoren: Basieren auf der Messung der Laufzeit von Schallimpulsen [Her+12, S. 443].

1.2. Definition und Funktion: Elektromechanischer Endschalter

Ein elektromechanischer Endschalter ist ein kontaktbehafteter Positionssensor. Ein solcher Schalter „hat die Aufgabe, das Erreichen einer bestimmten Position (Endlage) zu melden oder einen Schaltvorgang auszulösen“ [Her+12, S. 435]. Dabei wird durch das Betätigen eines mechanischen Elements, wie beispielsweise eines Hebels oder Tasters, ein elektrischer Kontakt geöffnet oder geschlossen [Her+12, S. 120–122].

Technische Spezifikationen (Beispiel Creality): Bei dem vorliegenden Bauteil handelt es sich um einen „originalen Endschalter von Creality 3D, der bei fast allen FDM Druckern von Creality 3D verbaut ist“ [3dj].

1.3. Specification

- Kompatibilität: Geeignet für die X-, Y- oder Z-Achse von Modellen wie der Ender-3 oder CR-10 Serie [3dj][Bas].
- Hersteller: Creality
- Hersteller SKU: 4004180004
- Gewicht: 7.3g
- Abmaße: 28mm x 20mm x 9mm

- Auslöseweg: 4mm

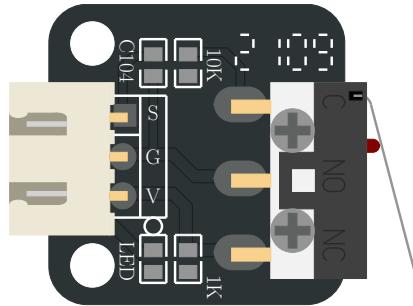


Abbildung 1.1.: Darstellung des im Projekt verwendeten Endschalters

1.4. Verwendung und Schaltfunktionen

Mechanische Endschalter werden zur Überwachung von Bewegungsabläufen eingesetzt. Dabei können verschiedene logische Funktionen realisiert werden:

- Schließer: Der Kontakt wird bei Betätigung geschlossen.
- Öffner: Der Kontakt wird bei Betätigung unterbrochen.
- Antivalenter Sensor: Dieser verfügt über zwei Ausgänge, wobei „stets ein Kontakt geschlossen und der andere offen“ ist [Her+12, S. 435]. Dies ermöglicht neben der reinen Schaltfunktion auch eine „Fehlererkennung und Diagnose durch die SPS“ [Her+12, S. 435–436].

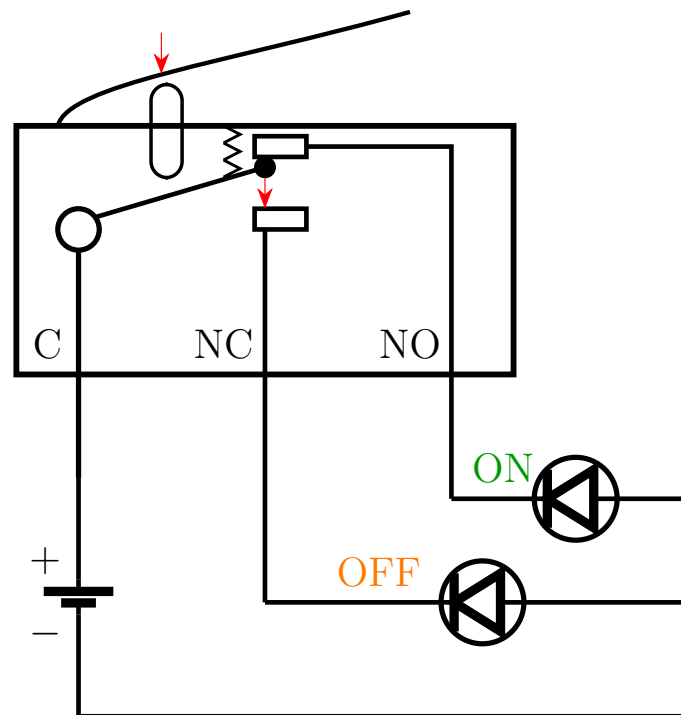


Abbildung 1.2.: Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO

Haupteinsatzgebiete:

- Endstopp-Erkennung: Schutz der Maschine vor dem Überfahren der mechanischen Grenzen.
- Referenzschalter (Homing): Der Schalter dient als „Referenzschalter“, um den Nullpunkt einer Achse (z. B. beim 3D-Drucker) zu kalibrieren [Bas].

1.5. Analyse: Vor- und Nachteile

Vorteile:

- Wirtschaftlichkeit: Sehr kostengünstig (ca. 4-5 € pro Stück) [3DJake, Bastelgarage].
- Einfachheit: Direkte Erzeugung eines binären Signals ohne aufwändige Auswerteelektronik.
- Robustheit gegen EMV: Unempfindlich gegenüber elektromagnetischen Störungen, da kein Oszillator wie bei induktiven Sensoren benötigt wird.

Nachteile:

- Mechanischer Verschleiß: Durch den physischen Kontakt unterliegen die Bauteile einer Abnutzung.
- Geschwindigkeit: Mechanische Kontakte können „prellen“ und sind langsamer als elektronische Lösungen [Her+12, S. 444].
- Technologische Verbreitung: Obwohl immer noch sehr viele mechanische Endschalter eingesetzt werden, ist der berührungslose Endschalter (Sensorschalter) in nahezu allen Bereichen als Standard-Bauelement zu finden [Her+12, S. 435].

1.6. Ausblick: Intelligente und vernetzte Sensorik

Die Zukunft der Positionserfassung liegt in der bidirektionalen Kommunikation. Ein moderner Standard ist hierbei IO-Link, welcher eine „feldbusunabhängige Kommunikation zwischen Sensoren und Aktoren und der Maschinensteuerung“ beschreibt [Her+12, S. 510].

Während klassische Endschalter nur „1 Bit“ an Informationen liefern (Ein/Aus), können intelligente Systeme über IO-Link zusätzlich Servicedaten übertragen, „die zur Einstellung oder Diagnose eines Devices führen“ [Her+12, S. 512].

Dies ermöglicht eine „vorbeugende Instandhaltung“, indem beispielsweise Verschleiß gemeldet wird, bevor es zum Maschinenausfall kommt [Her+12, S. 471, 512].

1.7. Library

Für die Verwendung des Endschalters werden keine zusätzlichen Bibliotheken benötigt. Es werden Funktionen aus der Arduino-Standardbibliothek verwendet:

- `pinMode()`: Konfiguriert einen Pin als Ein- oder Ausgang.
- `digitalRead()`: Liest den aktuellen Zustand eines digitalen Eingangs.
- `digitalWrite()`: Setzt einen digitalen Ausgang auf HIGH oder LOW.
- `Serial.begin()`: Initialisiert die serielle Kommunikation.
- `Serial.print()` / `Serial.println()`: Gibt Daten im Serial Monitor aus.
- `delay()`: Erzeugt eine zeitliche Verzögerung im Programmablauf.

1.8. Beispiel

Dieses Beispiel zeigt, wie der Endschalter über einen digitalen Eingang ausgelesen werden kann. Dabei wird der interne Pull-Up-Widerstand verwendet. Der Zustand des Schalters wird im Serial Monitor angezeigt und ändert sich beim Betätigen von HIGH auf LOW.

1.8.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND.

1.8.2. Erklärung des Beispiel-Codes

Nach der Initialisierung durch die Funktion `setup()` wird das Programm durch die Funktion `loop()` kontinuierlich ausgeführt. Diese Funktion stellt eine Endlosschleife dar, in der der Zustand des Endschalters bei jedem Durchlauf überprüft wird.[Ardj; Ardg]

Der verwendete Pin wird mit der Funktion `pinMode()` als Eingang oder Ausgang konfiguriert. [Ardi] Dabei wird der Modus `INPUT_PULLUP` verwendet, um den internen Pull-Up-Widerstand zu aktivieren.[Arda] Der interne Pull-Up-Widerstand sorgt dafür, dass am Eingang im Ruhezustand ein definiertes HIGH-Signal anliegt. Innerhalb der `loop()`-Funktion wird der Zustand des Endschalters mit der Funktion `digitalRead()` ausgelesen. Der gelesene Wert wird in einer Variablen gespeichert und über die serielle Schnittstelle ausgegeben.[Arde] Die Funktionen `Serial.print()` und `Serial.println()` werden verwendet, um den aktuellen Zustand des Eingangs im Serial

Monitor darzustellen.[Ardc] Durch die Verwendung des internen Pull-Up-Widerstands ergibt sich eine invertierte Logik. Im nicht betätigten Zustand des Endschalters liegt ein HIGH-Signal am Eingang an. Wird der Endschalter betätigt liegt ein LOW-Signal an. Zur besseren Lesbarkeit der Ausgabe wird eine Verzögerung mit der Funktion `delay()` erzeugt.[Ardd]

Listing 1.1.: Einfaches Beispiel zum Lesen eines mechanischen Endschalters

```

1000 /**
      * @file MicroSwitch_BasicRead.ino
1002  * @brief Basic example for reading a micro switch.
      */
1004 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
      switch. */

1006 /**
      * @brief Initializes the serial interface and the input pin.
1008  */
      void setup()
1010 {
      Serial.begin(115200);
1012   pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
      }

1014 /**
      * @brief Reads the switch state and prints it to the serial monitor.
1016  */
      void loop()
1018 {
1020   int switchState = digitalRead(MICRO_SWITCH_PIN);

1022   Serial.print("Micro switch state: ");
      Serial.println(switchState);

1024   delay(200);
1026 }

```

../Code/MicroSwitchBasicRead/MicroSwitchBasicRead.ino

1.9. Justage

Die Justage (auch "Justierung") ist eine "Tätigkeit, welche ein Messgerät in einen gebrauchstauglichen Betriebszustand versetzt. Sie kann manuell, halb automatisch oder automatisch erfolgen." [Hel, S. 17] Sie kann auch als das "abgleichen des Nullpunktes und der Kennliniensteigung" [Ber24, S. 17] verstanden werden. Da der verwendete Endschalter nur die analogen Werte OPEN = 0 und CLOSED = 1 detektieren kann, sodass keine Kennlinie im herkömmlichen Sinne vorliegt, liegt die anwendungsspezifische Justage-Aufgabe darin, den gesamten Endschalter physisch in seiner Position am Rahmengestell so auszurichten, dass er die entsprechende bewegliche Achse im Rahmen einer Referenzfahrt in ihrer Bewegung eingrenzt, aber gleichzeitig im Normalbetrieb den erforderlichen Bauraum nicht künstlich einschränkt.

1.10. Tests

1.10.1. Einfacher Funktions-Test

Im einfachsten Anwendungstest wird der Endschalter verwendet, um eine LED zu steuern. Hierbei wird der Zustand des Schalters ausgewertet und auf einen Ausgang übertragen. Wird der Endschalter betätigt, schaltet sich die LED ein. Im nicht betä-

tigten Zustand bleibt die LED ausgeschaltet. Durch diesen Test kann die Funktion des Sensors sowie die Verarbeitung des Signals im System überprüft werden.

1.10.2. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND. Die LED wird an einen digitalen Ausgang (z. B. D13) angeschlossen.

Beim Betätigen des Endschalters wird die LED eingeschaltet, im nicht betätigten Zustand bleibt sie ausgeschaltet.

1.10.3. Einfacher Test-Code

Die Funktion `loop()` läuft dauerhaft und liest den Zustand des Mikroschalters ein. Da `INPUT_PULLUP` verwendet wird, bedeutet `LOW` = gedrückt. Ist der Schalter gedrückt, wird die LED eingeschaltet, sonst ausgeschaltet.

Listing 1.2.: An- und Ausschalten einer LED mittels Endschalter

```

1000 /**
1001  * @file MicroSwitch_LEDExample.ino
1002  * @brief Simple application example using a micro switch and an LED.
1003  */
1004
1005 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1006      switch. */
1007 const int LED_PIN = 13;          /**< Digital output pin for the built-in
1008      LED. */
1009
1010 /**
1011  * @brief Initializes the input and output pins.
1012  */
1013 void setup()
1014 {
1015     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1016     pinMode(LED_PIN, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the LED depending on the micro switch state.
1021  */
1022 void loop()
1023 {
1024     int switchState = digitalRead(MICRO_SWITCH_PIN);
1025
1026     if (switchState == LOW)
1027     {
1028         digitalWrite(LED_PIN, HIGH);
1029     }
1030     else
1031     {
1032         digitalWrite(LED_PIN, LOW);
1033     }
1034 }

```

../Code/MicroSwitchLEDExample/MicroSwitchLEDExample.ino

1.11. Einfache Anwendung

Das Programm führt eine Referenzfahrt mit einem Schrittmotor aus.

Im `setup()` werden die Pins für den Treiber und den Endschalter eingerichtet. Danach wird der Motortreiber aktiviert und die Referenzfahrt gestartet.

Die Funktion `oneStep()` erzeugt einen einzelnen Schritt des Motors.

Mit `moveSteps()` kann der Motor eine festgelegte Anzahl an Schritten in eine bestimmte Richtung fahren.

In `homeAxis()` fährt der Motor so lange, bis der Endschalter gedrückt wird. Danach wartet das Programm kurz und fährt den Motor einige Schritte zurück.

1.11.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters wird an D2 angeschlossen, der zweite Anschluss liegt auf GND. Der Schrittmotortreiber wird über die Steuerpins STEP und DIR an digitale Ausgänge des Mikrocontrollers angeschlossen (z. B. STEP an D3 und DIR an D4).

Listing 1.3.: Referenzfahrt einer Achse mit Endschalter

```

1000 /**
      * @file referenzfahrt.ino
1002  * @brief Homing routine using a stepper motor, A4988 driver, and
      *       endstop.
      */
1004
1005 /** @brief STEP pin of the driver */
1006 const int stepPin = 2;
1007
1008 /** @brief Direction pin of the driver */
1009 const int dirPin = 5;
1010
1011 /** @brief Enable pin of the driver */
1012 const int enPin = 8;
1013
1014 /** @brief Endstop input pin */
1015 const int endstopPin = 9;
1016
1017 /** @brief Step delay in microseconds */
1018 const int stepDelayUs = 1000;
1019
1020 /** @brief Number of steps for the backoff movement */
1021 const int backoffSteps = 50;
1022
1023 /** @brief Initializes the hardware and starts the homing routine */
1024 void setup() {
1025     pinMode(stepPin, OUTPUT);
1026     pinMode(dirPin, OUTPUT);
1027     pinMode(enPin, OUTPUT);
1028     pinMode(endstopPin, INPUT_PULLUP);
1029
1030     Serial.begin(115200);
1031
1032     digitalWrite(enPin, LOW);
1033     digitalWrite(dirPin, LOW);
1034
1035     homeAxis();
1036 }
1037
1038 /** @brief Main loop (unused in this example) */
1039 void loop() {
1040 }
1041
1042 /** @brief Executes a single motor step */
1043 void oneStep() {
1044     digitalWrite(stepPin, HIGH);
1045     delayMicroseconds(stepDelayUs);
1046     digitalWrite(stepPin, LOW);

```

```
1048     delayMicroseconds(stepDelayUs);
1050 }
1051 /**
1052  * @brief Moves the motor by a defined number of steps
1053  * @param steps Number of steps to execute
1054  * @param dir Direction of movement (LOW/HIGH)
1055  */
1056 void moveSteps(int steps, bool dir) {
1057     digitalWrite(dirPin, dir);
1058     for (int i = 0; i < steps; i++) {
1059         oneStep();
1060     }
1061 }
1062 /** @brief Performs the homing sequence and backoff movement */
1063 void homeAxis() {
1064     while (digitalRead(endstopPin) == HIGH) {
1065         oneStep();
1066     }
1067
1068     delay(200);
1069     moveSteps(backoffSteps, HIGH);
1070 }
```

../Code/referenzfahrt/referenzfahrt.ino

1.12. Weiterführende Literatur

- Zielke, Dirk: *Mikrosysteme*. Springer, 2025. [Zie25] Das Buch behandelt die Grundlagen und Anwendungen von Mikrosystemen und bietet einen Überblick über Aufbau, Funktion und Einsatz moderner Sensorsysteme.
- Tränkler, Hans-Rolf und Reindl, Leo: *Sensortechnik*. Springer, 2014. [TR14] Das Buch behandelt grundlegende Prinzipien der Sensortechnik und beschreibt verschiedene Sensorarten sowie deren Einsatz in technischen Systemen.

2. Druckknopf

2.1. General

Ein elektrischer Druckknopf, auch Drucktaster genannt, ist ein elektromechanisches Bedienelement, mit dem ein Stromkreis durch Drücken geöffnet oder geschlossen wird. Je nach Ausführung kann er als Schließer oder Öffner arbeiten. Nach dem Loslassen kehrt ein Taster wieder in seine Ausgangsstellung zurück. [Rs2]

2.2. Joy IT

Beim Joy-IT Button-Micro handelt es sich um einen kompakten Micro-Arcadebutton mit Mikroschalter. Er besteht aus Nylon, ist in mehreren Farben erhältlich und für eine Belastungsgrenze von 12 V ausgelegt.[Joy]

2.3. Spezifikationen

- Abmessungen: Ø 32.8 x 29.9 mm
- Belastungsgrenze: 10 mA / 12 V DC [Joy]
- Schaltkreis

2.3.1. Functions

Für die Verwendung eines einfachen Druckknopfes werden keine zusätzlichen Bibliotheken benötigt. Die Abfrage kann direkt mit den Standardfunktionen der Arduino IDE erfolgen.

2.4. Beispiel Code

2.4.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Die LED bekommt einen 220 Ohm oder 330 Ohm Vorwiderstand. Der lange LED-Anschluss ist die Anode (+) und kommt über den Widerstand an Pin 13. Der kurze LED-Anschluss ist die Kathode (-) und kommt an GND.

Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()` legt fest, ob ein Pin als Eingang oder Ausgang verwendet wird. [Ardi]
- `INPUT_PULLUP` aktiviert den interne Pull-up-Widerstand des Arduino. [Arda]
- `digitalRead()` liest den Zustand des Druckknopfes aus. [Arde]
- `digitalWrite()` wird die LED ein- oder ausgeschaltet. [Ardf]

2.4.2. Code

Listing 2.1.: Led mit Druckknopf

```

1000 /**
1001  * @file ledmitdruckknopf.ino
1002  * @brief Schaltet eine LED ein, solange ein Druckknopf gedrueckt wird.
1003  *
1004  * Der Druckknopf ist zwischen Pin 2 und GND angeschlossen.
1005  * Die LED ist mit einem 220 Ohm oder 330 Ohm Vorwiderstand an Pin 13
1006  * angeschlossen.
1007  * Der lange LED-Anschluss ist die Anode (+), der kurze Anschluss ist
1008  * die Kathode (-).
1009  */
1010
1011 const int druckknopfPin = 2;    ///< Pin, an dem der Druckknopf
1012      angeschlossn ist
1013 const int ledPin = 13;          ///< Pin, an dem die LED angeschlossen ist
1014
1015 /**
1016  * @brief Wird einmal beim Start des Arduino ausgefuehrt.
1017  *
1018  * Hier werden die Pins als Ein- oder Ausgang festgelegt.
1019  */
1020 void setup() {
1021     pinMode(druckknopfPin, INPUT_PULLUP); ///< Druckknopfeingang mit
1022     internem Pull-up
1023     pinMode(ledPin, OUTPUT);              ///< LED-Pin als Ausgang
1024 }
1025
1026 /**
1027  * @brief Hauptprogramm, das dauerhaft wiederholt wird.
1028  *
1029  * Wenn der Druckknopf gedrueckt wird, wird die LED eingeschaltet.
1030  * Wird der Druckknopf losgelassen, wird die LED ausgeschaltet.
1031  */
1032 void loop() {
1033     int druckknopfStatus = digitalRead(druckknopfPin); ///< Aktueller
1034     Zustand des Druckknopfes
1035
1036     if (druckknopfStatus == LOW) {
1037         digitalWrite(ledPin, HIGH); ///< LED einschalten
1038     } else {
1039         digitalWrite(ledPin, LOW);  ///< LED ausschalten
1040     }
1041 }

```

../Code/ledmitdruckknopf/ledmitdruckknopf.ino

2.5. Einfacher Testcode

2.5.1. Manual

Der Taster wird mit einem Anschluss an GND und mit dem anderen an Pin 2 angeschlossen. Im seriellen Monitor der Arduino IDE wird dann angezeigt, ob der Druckknopf gedrückt ist oder nicht.

Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `INPUT_PULLUP`
- `digitalRead()`
- `Serial.begin()` Startet die serielle Verbindung zum Computer.[Ardb]

- `Serial.println()` Gibt den Wert 1 oder 0 im seriellen Monitor aus. [Ardc]
- `delay()` Wartet kurz, damit die Ausgabe lesbar bleibt. [Ardd]

2.5.2. Code

Listing 2.2.: Druckknopf Test

```

1000 /**
1001  * @file druckknopfrtest.ino
1002  * @brief Testet einen Druckknopf und gibt 0 oder 1 im seriellen Monitor
1003  *       aus.
1004  *
1005  * Der Druckknopf ist zwischen Pin 2 und GND angeschlossen.
1006  * Es wird der interne Pull-up-Widerstand verwendet.
1007 */
1008 const int druckknopfPin = 2; ///< Pin, an dem der Druckknopf
1009                               ///< angeschlossen ist
1010 /**
1011  * @brief Wird einmal beim Start des Arduino ausgeführt.
1012  *
1013  * Der Druckknopf-Pin wird als Eingang mit internem Pull-up-Widerstand
1014  * festgelegt.
1015  * Die serielle Ausgabe wird gestartet.
1016 */
1017 void setup() {
1018     pinMode(druckknopfPin, INPUT_PULLUP); ///< Eingang mit internem Pull-
1019     up
1020     Serial.begin(115200);                  ///< Startet die serielle
1021     Verbindung
1022 }
1023 /**
1024  * @brief Hauptprogramm, das dauerhaft wiederholt wird.
1025  *
1026  * Gibt 1 aus, wenn der Druckknopf gedrueckt ist.
1027  * Gibt 0 aus, wenn der Druckknopf nicht gedrueckt ist.
1028 */
1029 void loop() {
1030     int druckknopfStatus = digitalRead(druckknopfPin); ///< Liest den
1031     Zustand des Druckknopf
1032     if (druckknopfStatus == LOW) {
1033         Serial.println(1); ///< Druckknopf gedrueckt
1034     } else {
1035         Serial.println(0); ///< Druckknopf nicht gedrueckt
1036     }
1037     delay(200); ///< Kurze Pause fuer eine lesbare Ausgabe
1038 }

```

../Code/druckknopftest/druckknopftest.ino

2.6. Einfache Anwendung

Interrupt? Für Pause oder Start und mehrere LEDS gehen nach einander an.

2.6.1. Manual

Erklärung: In diesem Programm werden drei Druckknöpfe verwendet: Start, Pause und Stop. Der Start-Druckknopf startet eine LED-Abfolge, bei der fünf LEDs nacheinander

aufleuchten. Der Pause-Druckknopf hält die Abfolge an und setzt sie beim erneuten Drücken fort. Der Stop-Druckknopf beendet die Abfolge und schaltet alle LEDs aus. Anschluss: Die drei Druckknöpfe werden jeweils mit einem Anschluss an GND angeschlossen. Die anderen Anschlüsse kommen an Pin 2, 3 und 4. Die LEDs werden jeweils mit 220 Ohm oder 330 Ohm Vorwiderstand an Pin 8 bis 12 angeschlossen. Der lange Anschluss der LED kommt zum Arduino-Pin, der kurze Anschluss zu GND. Dabei werden folgende Arduino-Standardfunktionen verwendet:

- `pinMode()`
- `digitalRead()`
- `digitalWrite()`
- `digitalWrite()`
- `millis()` Liefert die Zeit seit dem Start des Arduino in Millisekunden. [Ardh]
- `delay()`

2.6.2. Code

Listing 2.3.: Druckknöpfe Led Abfolge

```

1000 /**
1001  * @file druckknoepfedabfolge.ino
1002  * @brief Steuert eine LED-Abfolge mit drei Druckknoepfen.
1003  *
1004  * Mit dem Start-Druckknopf wird die LED-Abfolge gestartet.
1005  * Mit dem Pause-Druckknopf wird die Abfolge pausiert oder fortgesetzt.
1006  * Mit dem Stop-Druckknopf wird die Abfolge beendet und alle LEDs werden
1007  *   ausgeschaltet.
1008  */
1009
1010 const int startdruckknopf = 2; ///< Eingangspin fuer den Start-
1011     Druckknopf
1012 const int pausedruckknopf = 3; ///< Eingangspin fuer den Pause-
1013     Druckknopf
1014 const int stopdruckknopf = 4; ///< Eingangspin fuer den Stop-
1015     Druckknopf
1016
1017 const int ledPins[] = {8, 9, 10, 11, 12}; ///< Ausgangspins fuer die
1018     LEDs
1019 const int anzahlLeds = 5;                ///< Anzahl der
1020     angeschlossenen LEDs
1021
1022 bool programmLaeuft = false; ///< Gibt an, ob die LED-Abfolge aktiv ist
1023 bool programmPause = false; ///< Gibt an, ob die LED-Abfolge pausiert
1024     ist
1025
1026 int aktuelleLed = 0;                ///< Nummer der aktuell
1027     eingeschalteten LED
1028 unsigned long letzteZeit = 0;        ///< Zeitpunkt der letzten LED-
1029     Umschaltung
1030 const unsigned long intervall = 500; ///< Zeitabstand zwischen den LEDs
1031     in Millisekunden
1032
1033 bool letzterPauseStatus = HIGH; ///< Vorheriger Zustand des Pause-
1034     Druckknopfs
1035
1036 /**
1037  * @brief Wird einmal beim Start des Arduino ausgefuehrt.
1038  *
1039  * Die Druckknoepfe werden als Eingange mit internem Pull-up-Widerstand
1040  * festgelegt.

```

```

1030  * Die LED-Pins werden als Ausgaenge festgelegt.
1031  */
1032  void setup() {
1033      pinMode(startdruckknopf, INPUT_PULLUP); ///< Start-Druckknopf als
1034      Eingang
1035      pinMode(pausedruckknopf, INPUT_PULLUP); ///< Pause-Druckknopf als
1036      Eingang
1037      pinMode(stopdruckknopf, INPUT_PULLUP); ///< Stop-Druckknopf als
1038      Eingang
1039
1040      for (int i = 0; i < anzahlLeds; i++) {
1041          pinMode(ledPins[i], OUTPUT); ///< LED-Pin als Ausgang festlegen
1042      }
1043
1044      alleLedsAus();
1045  }
1046
1047  /**
1048  * @brief Hauptprogramm, das dauerhaft wiederholt wird.
1049  *
1050  * Es prueft die drei Druckknoepfe und steuert je nach Eingabe
1051  * den Ablauf der LEDs.
1052  */
1053  void loop() {
1054      if (digitalRead(startdruckknopf) == LOW) {
1055          programmLaeuft = true;
1056          programmPause = false;
1057      }
1058
1059      if (digitalRead(stopdruckknopf) == LOW) {
1060          programmLaeuft = false;
1061          programmPause = false;
1062          aktuelleLed = 0;
1063          alleLedsAus();
1064      }
1065
1066      bool aktuellerPauseStatus = digitalRead(pausedruckknopf); ///<
1067      Aktueller Zustand des Pause-Druckknopfs
1068
1069      if (letzterPauseStatus == HIGH && aktuellerPauseStatus == LOW) {
1070          programmPause = !programmPause;
1071          delay(200); ///< Einfache Entprellung des Druckknopfs
1072      }
1073
1074      letzterPauseStatus = aktuellerPauseStatus;
1075
1076      if (programmLaeuft && !programmPause) {
1077          ledAbfolge();
1078      }
1079  }
1080
1081  /**
1082  * @brief Schaltet alle LEDs aus.
1083  *
1084  * Alle LED-Pins werden auf LOW gesetzt.
1085  */
1086  void alleLedsAus() {
1087      for (int i = 0; i < anzahlLeds; i++) {
1088          digitalWrite(ledPins[i], LOW);
1089      }
1090  }
1091
1092  /**
1093  * @brief Laesst die LEDs nacheinander aufleuchten.
1094  *
1095  * Nach Ablauf des festgelegten Intervalls wird die aktuelle LED
1096  * ausgeschaltet
1097  * und die naechste LED eingeschaltet.
1098  */

```

```
void ledAbfolge() {  
1094   unsigned long aktuelleZeit = millis(); ///  
        Aktuelle Laufzeit des  
        Arduino in Millisekunden  
  
1096   if (aktuelleZeit - letzteZeit >= intervall) {  
1098       letzteZeit = aktuelleZeit;  
  
        alleLedsAus();  
1100       digitalWrite(ledPins[aktuelleLed], HIGH);  
  
1102       aktuelleLed++;  
  
1104       if (aktuelleLed >= anzahlLeds) {  
1106           aktuelleLed = 0;  
        }  
1108   }
```

../Code/druckknoepfedabfolge/druckknoepfedabfolge.ino

2.7. Further Readings

Teil III.

Domain Knowledge - Tools

Teil IV.

Development

Teil V.

Monitoring

Teil VI.

**Verification - Evaluation -
Conclusion**

Teil VII.

Anhang

Literaturverzeichnis

- [3dj] *Creality Endschalter*. 2026. URL: <https://www.3djake.de/creality-3d-drucker-ersatzteile/endschalter> (besucht am 26.04.2026).
- [Arda] *Digital Pins - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/variablen/constants/inputOutputPullup/> (besucht am 25.04.2026).
- [Ardb] *Serial.begin()* - *Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/communication/serial/begin/> (besucht am 10.05.2026).
- [Ardc] *Serial.print()* - *Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/communication/serial/print/> (besucht am 25.04.2026).
- [Ardd] *delay()* - *Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/time/delay/> (besucht am 25.04.2026).
- [Arde] *digitalRead()* - *Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/digitalread/> (besucht am 25.04.2026).
- [Ardf] *digitalWrite()* - *Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/funktionen/digital-io/digitalwrite/> (besucht am 10.05.2026).
- [Ardg] *loop()* - *Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/loop/> (besucht am 25.04.2026).
- [Ardh] *millis()* - *Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/funktionen/time/millis/> (besucht am 10.05.2026).
- [Ardi] *pinMode()* - *Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/pinMode/> (besucht am 25.04.2026).
- [Ardj] *setup()* - *Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/setup/> (besucht am 25.04.2026).
- [Bas] *Creality Endschalter*. 2026. URL: <https://www.bastelgarage.ch/creality-endschalter-limit-switch> (besucht am 26.04.2026).
- [Ber24] H. Bernstein. *Messelektronik und Sensoren. Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. 2. Aufl. Springer Vieweg Wiesbaden, 25. Jan. 2024. ISBN: 978-3-658-38929-1. DOI: <https://doi.org/10.1007/978-3-658-38929-1>.
- [Hel] W. Helbig. *Praxiswissen in der Messtechnik. Arbeitsbuch für Techniker, Ingenieure und Studenten*.
- [Her+12] E. Hering u. a. *Elektrotechnik und Elektronik für Maschinenbauer*. Bd. 2. Springer, 14. Feb. 2012. ISBN: 978-3-642-12881-3.
- [Joy] *ARKADE-BUTTONS MIT MIKROSCHALTER*. 2024.
- [Rs2] *Drucktaster von Eaton: Ein Leitfaden*. 2026. URL: <https://de.rs-online.com/web/content/discovery-portal/produktatgeber/drucktaster-leitfaden> (besucht am 09.05.2026).

- [TR14] H.-R. Tränkler und L. Reindl, Hrsg. *Sensortechnik: Handbuch für Praxis und Wissenschaft*. 2. Aufl. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, S. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: **10.1007/978-3-642-29942-1**.
- [Zie25] D. Zielke. *Mikrosysteme*. Bd. 1. Springer, 5. März 2025. ISBN: 978-3-662-71232-0.